CERTIQS-KEY

An overview of the contracts

The contracts of CERTIQS-KEY state and prove what it takes to compute, from a declared description of a quantum key distribution protocol and its devices, a fixed-length key length together with a security parameter ε, the sum of the privacy-amplification and error-verification errors and twice the authentication error. The protocol is ε-secure in the composable sense whenever every assumption the key length rests on holds of the devices, the laboratories and the run and, for a number that is not certified, the bound the run computed lies below the value it bounds, a condition no assumption covers (*What a key length is about* below). Each of those assumptions is named. There is one contract per *core module*, which holds the claims every protocol needs in one area, and the contract includes the module's *submodules*, small bundles of claims for one device, source model, squasher or statistical method that a protocol uses only when it needs them. A *claim* is one numbered definition, theorem, lemma, assumption, requirement (an obligation on the implementation) or other statement, and its identifier, such as meat-core/thm:keylength, gives its module, a prefix naming its kind, and a name. This overview says what the contracts establish together, explains the terms they use throughout, says how the modules fit and how a contract is laid out, and suggests four reading paths; the main results of every contract and the module map close it.

What the contracts establish

Consider a fixed-length protocol of n rounds. In each round Alice prepares a signal, a receiver measures it, and the parties announce public information, sift and record a raw key. After the last round the protocol accepts when the observed frequencies of the acceptance outcomes lie in an *acceptance set* fixed before the run; on acceptance, error correction, error verification and privacy amplification turn the raw key into the key. The user declares the protocol and its devices in a *protocol specification* (Definition "Protocol specification" (protocol-compiler/def:spec)). The contracts replace Alice's source and the receiver by finite models and compile the result into a *protocol declaration* (Definition "Protocol declaration" (protocol-compiler/def:declaration)): Alice's system and its marginal, a finite measured system split into sectors, the measurements of both parties, what each party reads and announces, the map from the announcements to the acceptance outcome, the raw-key map with the party that holds the key, and a list of constraint entries, possibly empty, which the next section explains.

From the declaration the contracts derive the channel of one round (Theorem "The declaration generates a protocol channel" (meat-core/thm:

protocol-channel)). The published theorem of marginal-constrained entropy accumulation, MEAT (Fact "Marginal-constrained entropy accumulation, sequential form" (meat-core/fact:meat)), applied to that channel, bounds the Rényi entropy of order $\alpha > 1$ of the accepted raw key, given everything the adversary holds, from below by n times a *single-round rate* less the penalty $\alpha/(\alpha-1)\cdot\log_2(1/\Pr[\text{accept}])$, against every adaptive attack whose quantum memory is finite-dimensional, of any dimension (Theorem "Accumulated entropy bound under adaptive attacks" (meat-core/thm:accumulated-entropy)); the security of the key extends, by a limit over those dimensions, to an adversary whose memory has any dimension and who receives the signals as they are released (Corollary "Security of the protocol and its net key" (meat-core/cor:composed-security), through Lemma "Approximating an attack by all-at-once attacks with finite memory" (meat-core/lem:memory-lift)). The single-round rate is an infimum, over the distributions in the acceptance set and the single-round states that meet the declared marginal, of a divergence penalty plus the conditional entropy of the round's key (Definition "Constrained target set and single-round rate" (meat-core/def:hqkd-list)). The contracts bound it from below by the value $\underline{h}_\alpha$ of a finite convex program (Theorem "Reduction of the rate to the convex program" (meat-core/thm:reduction)); the summaries, which are plain text, write $\underline{h}_\alpha$ as h_low. For $\alpha \in (1,2)$ a key of length

$$\ell \;\le\; n\,\underline{h}_\alpha - \lambda_{\mathrm{EC}} - \left\lceil \log_2(1/\varepsilon_{\mathrm{EV}}) \right\rceil - \alpha/(\alpha-1)\cdot\log_2(1/\varepsilon_{\mathrm{PA}}) + 1$$

is then $\varepsilon_{\mathrm{PA}}$-secret (Theorem "Secrecy of the key from the accumulated entropy" (meat-core/thm:keylength)) and $\varepsilon_{\mathrm{EV}}$-correct (Lemma "Correctness from error verification" (meat-core/lem:ev-correctness)) on the accepted state, λ_{EC} being the length in bits of the error-correction transcript of the post-processing model (Definition "Fixed-length post-processing model" (meat-core/def:postprocessing-model)), fixed before the run (Assumption "Protocol parameters fixed before the run" (meat-core/ass:fixed)). Both statements are about the accepted state left unnormalised, that is, counted over all runs, aborted ones included: secrecy bounds the trace distance of the key jointly with the event of acceptance from a uniform key independent of everything the adversary holds, and correctness bounds the probability that the run accepts and the two keys differ. Counted that way the penalty in $\Pr[\text{accept}]$ cancels, which is why no such term appears in ℓ. Both are stated in the setting where every classical message arrives as sent; over an authentication scheme within $\varepsilon_{\mathrm{auth}}$ that consumes k_{auth} bits of pre-shared key (Assumption "Authentication of the classical channel by a keyed scheme" (meat-core/ass:auth)), the protocol is $(\varepsilon_{\mathrm{PA}} + \varepsilon_{\mathrm{EV}} + 2\varepsilon_{\mathrm{auth}})$-secure and a run in which both parties output a key gains $\ell - k_{\mathrm{auth}}$ bits (Corollary "Security of the protocol and its net key" (meat-core/cor:composed-security)), and the security parameters of separate executions add (Corollary "Security parameters of separate executions add" (meat-core/cor:separate-executions)). Any

lower bound $h \le \underline{h}_\alpha$ may replace $\underline{h}_\alpha$ in the displayed bound on ℓ through $\max\{0, h\}$, with the same security statement (Corollary "Key length from any lower bound on the value" (meat-core/cor:keylength-bound)); this is how a computed number becomes a key length. The constant $+1$ is the announced outcome of error verification at work: announcing it costs one bit and makes secrecy hold on the accepted state, where the derivation this bound generalises, with its secrecy conditioned on the acceptance test alone, carries $+2$; the published fixed-length statement reaches the accepted state with $+2$ by another route, which this module does not import (Remark "The constant in the key length and the source's convention" (meat-core/rem:plus2)). The bound is for an ideal privacy-amplification hash family (A-HASH); for a family that is only two-universal it is tightened by one further bit, $\ell \to \ell - 1$ (Requirement "Key-length penalty for a merely two-universal family" (meat-core/req:hash-penalty)).

What a key length is about. It is a statement about the declared model, and it holds of an apparatus only through the assumptions that the apparatus is that model exactly: a trusted source emits exactly the declared states (Assumption "Emission of the specified ensemble" (transmitter/ass:ens), or for a decoy source Assumption "Emission of the declared weak-coherent-pulse states" (transmitter/decoy/ass:wcp)), the receiver is one declared measurement (Assumption "Trusted and exactly characterised receiver" (receiver/ass:trust), Assumption "Specification faithful to the apparatus" (protocol-compiler/ass:spec)) and a declared parameter carries no uncertainty (Assumption "Exact declared parameters" (receiver/ass:exact)). No tolerance computed from the declared dimensions, alphabet sizes and α can stand in for exactness: two exactly admissible measurements within ϵ of each other, in operator norm outcome by outcome, can have rate bounds far apart (Proposition "Instability of the rate under measurement perturbations" (meat-core/prop:optol-nobound)). And what a run computes is a lower bound on $\underline{h}_\alpha$ only in the sense of its tier (*Three tiers* below): a *first-order* number, which a run reports unless its certified evaluation succeeds, is a lower bound in exact real arithmetic, and no claim bounds the rounding of the binary64 evaluation a run performs, nor does an assumption cover it (Remark "Exact real arithmetic in the first-order bound" (meat-core/frank-wolfe/rem:fo-arithmetic)): the security of a key length from a first-order number also needs the bound the run computed to lie below $\underline{h}_\alpha$, which no assumption covers and a run cannot check. A certified number removes that condition. Every assumption a key length rests on is named in (d) of the reading paths.

Exact decisions and named assumptions. Every condition on declared numbers is decided exactly. A declared number is a binary64 (double-precision) value, read as the dyadic rational it denotes. A sum that a decision reads is formed in a declared order and *rounds nowhere*: each of its binary64 additions is exact, so the computed sum is the exact sum (Lemma "Exactness of a sum that rounds nowhere" (foundations/lem:exact-accumulation)).

An operator family handed from one module to another is exactly admissible, a property of its declared numbers that the next section states (Requirement "Exact admissibility at every module boundary" (foundations/req:boundary-admission)). Where a number is only evaluated in floating point, as in the receiver's device model, the contract says so and asserts no enclosure of it (Definition "The evaluation model: floating point, enclosures and rigour" (receiver/def:evaluation-model)); the reductions module, which reads the receiver's output, rests on the attestation that the device facts the receiver states hold of its numbers as declared (Assumption "Supplier artefact whose claims hold of the device" (reductions/ass:recv)). What declared numbers cannot settle, such as that a source emits the declared states or that a receiver is exactly characterised, is an *assumption*: a checkable hypothesis, stated once and printed with what its failure puts at stake (secrecy, correctness or completeness) and whether the engine verifies it. Each claim names the assumptions it declares, and the claims it uses name theirs, so that a key length can be traced to every assumption behind it.

Three tiers. A key-rate number a run reports carries one of three tiers (Definition "The tiers of a reported key rate" (foundations/def:tiers)). An *estimate* is the value of the objective at the point an optimiser returned. At an exactly feasible point it bounds $\underline{h}_\alpha$ from above (Lemma "Upper estimate of the value at any feasible point" (meat-core/lem:estimate)), so it backs no key length. A *first-order* number is a lower bound on $\underline{h}_\alpha$: the best linearisation of the objective over the iterates of a Frank–Wolfe run. It lies below $\underline{h}_\alpha$ under the hypotheses of Theorem "The estimate and the first-order lower bound" (meat-core/frank-wolfe/thm:firstorder), with its two linear minima bounded from below rather than trusted: the acceptance-side one exactly, and the feasible-side one from the solver's dual in exact arithmetic (Lemma "A lower bound on the feasible-side minimum from any multipliers" (meat-core/frank-wolfe/lem:feasible-side-dual)), and a key length follows from it (Corollary "Key length from the first-order bound" (meat-core/frank-wolfe/cor:fo-keylength)). That theorem holds in exact real arithmetic. A run evaluates the objective and its gradient in binary64, and no claim bounds the rounding error of that evaluation (Remark "Exact real arithmetic in the first-order bound" (meat-core/frank-wolfe/rem:fo-arithmetic)): the key-length statement is about the exact first-order bound at the run's iterates, and no assumption covers the rounding. A *certified* number closes that gap at a run's final points: the same bound formed again from the declaration's exact data, every step exact or rigorously enclosed, only the gradient needing an enclosure (Lemma "The certified bracket from enclosures of the gradient" (meat-core/frank-wolfe/lem:certified-bracket)), and its key length computed exactly, its one logarithm enclosed until the integer part is decided (Corollary "Key length from the certified bracket" (meat-core/frank-wolfe/cor:certified-keylength)). A run reports it where its certified evaluation succeeds (Requirement "The certified evaluation of the first-order

bound" (meat-core/frank-wolfe/req:certified-evaluation)) and otherwise keeps its first-order number with the reasons; the declaration's modelling premises remain, and no premise about arithmetic is added.

The reference companion. Where a measurement was moved onto the binary64 lattice by a *snap* (*Emission* below) and the run holds a *reference* of the model it was declared from, exact or known through an entrywise enclosure (Definition "Reference of a measurement family, exact or enclosed" (foundations/def:exact-reference)), the engine may admit a *reference companion*: the declaration compiled with the reference in place of the snapped measurement (Proposition "The compiled reference companion is an admissible declaration" (meat-core/prop:reference-companion)). Where it admits one, the bound is formed on the companion, at points a search on the snapped declaration found, and its key length is that of the protocol whose declaration is the companion, the reference model's (Corollary "The first-order bound on the reference companion" (meat-core/frank-wolfe/cor:fo-reference), Corollary "Key length on a reference companion from the certified bracket" (meat-core/frank-wolfe/cor:certified-keylength-reference)); the companion's bracket can still be withdrawn, and its certified number can fall back to one on the snapped declaration (Requirement "The first-order bound evaluated on the reference companion" (meat-core/frank-wolfe/req:bracket-on-reference), Requirement "The certified evaluation of the first-order bound" (meat-core/frank-wolfe/req:certified-evaluation)), and the key length is then formed on the snapped declaration. Where the companion's bound stands, the run records the bound on the snapped declaration beside it; no key length is formed from that bound, which serves only diagnostics and the statistics check, and that check can only withhold a key. No claim relates the two bounds. A first-order number on a companion known through an enclosure is evaluated at the enclosure's centres, a displacement no claim bounds; the certified number is evaluated over the enclosure where every element the gradient reads has a rank fixed throughout it, and then holds of the companion itself (Remark "Exact real arithmetic in the first-order bound" (meat-core/frank-wolfe/rem:fo-arithmetic)). Where a run reads a moved measurement instead, its key length is about the moved data, and A-SNAP, that the device performs the moved measurement on the blocks the run record names, is a premise of it (Assumption "The device measures the snapped family where the run reads the snap" (foundations/ass:snapped-measurement)).

Scope. The contracts treat fixed-length protocols in which the same declaration describes every round, with devices characterised exactly: a receiver is one declared measurement rather than a family of possible devices, and a declared parameter carries no uncertainty. Variable-length protocols, sources correlated across rounds and uncertainty sets over devices lie outside them.

Terms used throughout

Each contract defines its terms where it first needs them. The terms below recur across the modules, and a reader meets them in the summaries of the main results before any definition.

Declared data; specification; declaration. Declared data are numbers a party states rather than computes: an *instance*, the party that fixes one protocol's data, or a module that hands data on as its own declaration (Definition "Declared data and the parties to a module boundary" (foundations/def:declared-data)). The protocol specification is what the user declares; the protocol declaration is the finite object the protocol compiler emits from it and the MEAT core reads. *Declaration* alone means data a party states as its own, so the output of a module is also called a declaration, as in "the emitted decoy declaration".

Emission; artefact; supplier. A module *emits* data when it hands them to another module, its *consumer*, as its own declaration. Before emitting it decides exactly that the data meet the conditions the consumer reads, and otherwise it refuses (Requirement "Exact verification before emission" (foundations/req:exact-emission)). A device module emits an *artefact*: its data with the records a consumer reads, such as a licence record and an assumption ledger naming the assumptions the data rest on. Where exact products have left the lattice of binary64 values a module emits on, the module may move its own data back onto the lattice by a *snap* (Definition "Snapping declared data onto a lattice" (foundations/def:snap)) and emit the moved data, which its theorems are then about. No claim relates a key length computed from the moved data to one computed from the data before the move: measurements close to each other entry by entry can have rate bounds far apart (Proposition "Instability of the rate under measurement perturbations" (meat-core/prop:optol-nobound)). A *supplier*, such as the receiver, decides its conditions in floating point, and its output becomes declared data only when an instance states it.

Sector map; exact admissibility. An operator on a system split into sectors is held as a block list, one matrix per sector, against a *sector map*: the ordered sector identifiers with their dimensions and a label saying what they are. Two sector maps *agree* when all of these coincide (Definition "Sector maps and block-list operators" (foundations/def:sector-map)). A family of block lists is *exactly admissible* when five conditions hold of its declared numbers: its sector map agrees with the consumer's; every block is exactly Hermitian; every block is exactly positive semidefinite; on each sector the blocks sum to the identity (or to a declared constant); and that sum, formed in the declared order, rounds nowhere (Definition "Exact admissibility of a declared operator family" (foundations/def:exact-admissibility)).

Source modes; source replacement. In *trusted-source mode* Alice keeps a system of her own that never reaches the adversary and that only her declared measurement touches (*retention*, Assumption "Retention of the system A in trusted-source mode" (transmitter/ass:src)), and her marginal on it is a declared state. A *source-replacement certificate*, checked exactly, then describes her preparation as that measurement on her retained system, and a consumer may fix her marginal; across rounds the marginal is a product, derived from the source's premises (Lemma "Product marginal of Alice's retained systems" (meat-core/lem:product-marginal)). In *untrusted-source mode* the adversary supplies Alice's system as well, as for an entanglement source outside the parties' control: no marginal is declared and no certificate is needed.

Squashing; spectral certificates. A *squashing channel* maps the physical measured system, whose photon number is unbounded, to a finite system, so that its adjoint takes every element of a finite *target* POVM to the physical element; the physical and the squashed protocol then produce the same output state. A squasher may also *carry operators* on the finite system whose *pull-backs*, their images under the adjoint, express facts about the device. A *spectral certificate* states that such a pull-back is equal to, at most or at least a multiple of the identity, whatever the input (Theorem "The attack-set licence" (reductions/thm:licence)). The squashers of the submodule wpfss (weight-preserving and flag-state squashers) carry the receiver's multiclick weights into the finite model in this way.

Constraint entries; attack-set licence. A *constraint entry* is an operator $\hat{O}$ on the finite measured system, a relation $\bowtie$ among $\leq$, $=$ and $\geq$, and a right-hand side, in the simplest form a real constant c; a squasher's carried operators with their spectral certificates yield such entries. A list of entries with real constants cuts out an *attack set*: the adversary's round channels $\mathcal{A}$ with $\mathcal{A}^\dagger(\hat{O} \otimes \mathbb{1}) \bowtie c\,\mathbb{1}$ for every entry, the identities acting on the other systems (Definition "Round model, attack sequences and attack sets" (foundations/def:round-model)). On a non-empty list the MEAT core minimises only over states that meet every entry, and the resulting *constrained* rate and value are never below the *unconstrained* ones, those of an empty list. Their use in a key length needs the *attack-set licence* (Assumption "Attack-set licence" (protocol-compiler/ass:atk)): the list is *licensed*, every right-hand side being a real constant, read as a multiple of Alice's marginal in trusted-source mode and as a number in untrusted-source mode, or the entry being party-local (Definition "Licensed entries and the party-local form" (reductions/def:licensed)); and every physical attack, once squashed, lies in the attack set. The reductions module proves the licence for entries that carry spectral certificates (Theorem "The attack-set licence" (reductions/thm:licence)). A *party-local* entry is scoped to one factor of a product measured system, such as one party's device.

Key-length eligibility. The protocol compiler's emission gates decide whether a well-formed declaration may back a key-length claim: its checks pass, the transmitter's licence record for the source-replacement certificate is present and re-decided where Alice's marginal has dimension two or more, and a non-empty constraint list is licensed and carries a record of its attack-set licence (Definition "Key-length eligibility" (protocol-compiler/def:eligible)). A declaration that fails a gate is refused, or emitted marked as not eligible. Eligibility is a property of the declaration and its records: passing the gates asserts nothing about the apparatus or about whether the physical attacks lie in the attack set (Theorem "The emission gates and their outcomes" (protocol-compiler/thm:gates)), which remain assumptions (Assumption "Specification faithful to the apparatus" (protocol-compiler/ass:spec), Assumption "Attack-set licence" (protocol-compiler/ass:atk)).

Acceptance set; honest model; completeness. The acceptance set is a closed convex subset of the probability simplex on the acceptance outcomes (Definition "Admissible acceptance set" (meat-core/def:admissible-acceptance-set)). The statistics module builds it from an *honest model*: one law of an honest round's acceptance outcome, or an *honest envelope*, a range of probabilities per outcome within which each honest round's law, given the rounds before it, stays. The *completeness budget* is the probability of aborting an honest run that the user allows. The honest model bears on completeness, never on the soundness of a key.

How the contracts fit

We describe the modules in the order in which the main results at the end list them: each comes after every module it builds on, that is, every module whose claims its own claims may use. Each item gives the contract's title, with the module's name in parentheses, then what the module builds on, what it establishes and its submodules.

Foundations: quantum-information preliminaries, number model, exact admissibility ar builds on no other module and is specific to no protocol. It states the quantum-information preliminaries the entropy optimisation is written in, the round model with its attack sets, and the number model: binary64 values, sums that round nowhere, exact admissibility (Definition "Exact admissibility of a declared operator family" (foundations/def:exact-admissibility)) and the rules of emission.

Receiver POVM (receiver) builds on foundations. It turns a declared passive optical receiver into the click-pattern POVM it performs, and into weights that bound its *multiclick element*, the sum of the POVM elements with two or more clicks, from below on every photon-number block above a cutoff (Theorem "Multiclick bounds carried by the emitted weights"

(receiver/thm:emitted-bound)). Its submodules add detector efficiency and dark counts (Theorem "Dark counts on a photon-number block-diagonal POVM" (receiver/detector-imperfections/thm:dark-counts)) and a catalogue of optical components (Theorem "Admissibility of the catalogue entries" (receiver/component-catalogue/thm:catalogue-admissibility)).

Transmitter source model (transmitter) builds on no other module. It describes what Alice prepares, the system she retains and the source mode, and in trusted-source mode the source-replacement certificate, whose conditions are finitely many exact decisions (Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency)). Its decoy submodule makes a phase-randomised weak-coherent-pulse source finite through one tagging channel (Proposition "Tagging channel for weights at or below the Poisson weights" (transmitter/decoy/prop:tagging)).

Proof reductions by squashing (reductions) builds on receiver, transmitter and foundations. It replaces the physical receiver, and in trusted-source mode the physical source as well, by finite models whose security transfers to the physical protocol with the same ε when the squashed inputs analysed contain the squashed image of every physical input (Theorem "Security transfer from a constrained squashed model" (reductions/thm:transfer)), and it emits them as a reduction chain with the constraint entries it certifies. Its submodule wpfss provides the flag-state and weight-preserving squashers (Proposition "Flag-state squashers: existence, chain and encoded weights" (reductions/wpfss/prop:chain)); its domination submodule relates a measurement family to one that dominates it.

Protocol compiler (protocol-compiler) builds on receiver, reductions, transmitter and foundations. It compiles a specification into the protocol declaration and decides exactly that the declaration is well formed, and its emission gates decide whether the declaration may back a key-length claim (Theorem "The emission gates and their outcomes" (protocol-compiler/thm:gates)).

MEAT key-rate engine (meat-core) builds on protocol compiler, reductions, transmitter and foundations. It turns the declaration and the acceptance set into the convex program and a key length, through entropy accumulation (Theorem "Accumulated entropy bound under adaptive attacks" (meat-core/thm:accumulated-entropy)) and the reduction of the single-round rate to that program (Theorem "Reduction of the rate to the convex program" (meat-core/thm:reduction)). Its Frank–Wolfe submodule supplies the first-order lower bound on the program's value (Theorem "The estimate and the first-order lower bound" (meat-core/frank-wolfe/thm:firstorder)).

Acceptance statistics (statistics) builds on reductions alone, for the as-

sumption that each round's public choices enter the test only through the recorded outcome (Assumption "Output-state test statistics" (reductions/ass:tsm)). Before the run it builds, from the honest model, the acceptance set the observed frequencies are tested against (Theorem "Validity of the admissible geometries" (statistics/thm:geometry-validation)). Its binomial-box submodule constructs a box and bounds the probability that an honest run fails the test (Theorem "The completeness bound" (statistics/binomial-box/thm:completeness-bound)), and its singleton submodule evaluates a one-point test exactly. Statistics and the MEAT core do not build on each other: the MEAT core assumes of its acceptance set what statistics proves of the sets it builds.

Data move towards the key length. The receiver hands its click-pattern POVM and its multiclick weights to the reductions module. The transmitter hands Alice's source, with its certificate, to the reductions module, the protocol compiler and the MEAT core. The reductions module emits a reduction chain in two segments: source links that replace Alice's signal states, then receiver links that replace the receiver's measurement, with the constraint entries the chain certifies. The protocol compiler joins that chain, the transmitter's artefact and the public part of the protocol into the protocol declaration. The statistics module builds the acceptance set on the declaration's acceptance outcomes, and the MEAT core computes the key length from the declaration and the acceptance set.

Not every protocol uses every module. An ideal qubit protocol declares its measurement directly and reaches the compiler through one identity link, with no squasher and no constraint entries; the compiler calls this direct route Band A (Theorem "The Band-A route" (protocol-compiler/thm:band-a)). In untrusted-source mode Alice's source carries no certificate.

How to read a contract

A contract is one PDF: the core module's mathematics in Part I, closed by its assumptions, the requirements on an implementation in Part II, then one part per submodule. A statements digest beside it gives every definition, assumption, result and requirement with the contract's numbers, without proofs, remarks, examples or technical steps. A claim's identifier names its module, with a submodule's path inside it (meat-core/frank-wolfe/thm:firstorder), then its kind: def a definition; thm, prop, lem and cor a theorem, proposition, lemma and corollary; fact a published result imported in the contracts' notation with its citation; ass an assumption; req a requirement; rem a remark; ex an example. An assumption states a checkable hypothesis, with what its failure puts at stake beside it, and its heading also shows a short id, such as A-SEQ, by which summaries name it.

Grey text is non-normative: the module overview, the section introductions, the grey italic summary line above each claim and the grey lead-in before it explain, and nothing in them is assumed or proved. The statements, set upright, the proofs and the assumptions are the contract. A main result, one of the few results a module exists to establish, is framed by a thin rule in the margin and has a bold heading. The front matter lists the main results with every assumption each rests on, and a table, *Assumptions at a glance*, of every assumption the contract states or its claims assume. Below each statement a small grey meta line lists what the claim uses, the assumptions it declares, the claims that use it, whether it is published, with its citation, or newly derived, its status and the code that implements it. Claims are numbered within their section on one counter, so "Proposition 7.3" names one claim whatever its kind; a reference to another contract gives the claim's kind, title and identifier, and opens that contract's PDF at the claim.

This overview explains in the same sense: where it and a contract differ, the contract's statement is what is claimed. Its last two sections are generated from the claim files. *Main results, module by module* gives each main result's number and title in its contract, followed by its summary line, a paraphrase in words set there in ordinary type, not the statement; the heading opens the statement. A reading path names a claim by the same title, so a path's entry and a main result match by title. *The module map* draws the relations of "How the contracts fit" as arrows, from each module to each module it builds on; a text rendering loses the arrows, and that section states the same relations in words.

Reading paths

Four paths lead through the claims. Each lists claims in the order we suggest reading them, with one line on what each adds to its title; a link opens the statement in its contract. To see why a key length is secure, read (a). To declare a new source or receiver, read (b), then (d). To choose an acceptance test and see where the finite-size terms come from, read (c). To audit a reported key length, read (d), then items 8 to 11 of (a) and the paragraphs above on what a key length is about, the three tiers and the reference companion.

(a) The key-rate chain

1. Definition "Protocol declaration" (protocol-compiler/def:declaration): the finite object the rest of the chain reads, whose parts "What the contracts establish" lists.

2. Theorem "Soundness, completeness and decidability of the validator" (protocol-compiler/thm:validator-soundness): the validator's checks, each decided exactly and sector block by sector block, establish that a declaration is well formed; only the check that exhibits a feasible state can refuse a well-formed declaration.
3. Theorem "The emission gates and their outcomes" (protocol-compiler/thm:gates): which well-formed declarations are eligible to back a key-length claim, a decision on the declaration that asserts nothing about the apparatus.
4. Theorem "The declaration generates a protocol channel" (meat-core/thm:protocol-channel): the channel of one round, with classical key, acceptance and announcement registers; its acceptance marginal is the statistics map, the affine map from a state to the distribution of the acceptance outcome.
5. Fact "Marginal-constrained entropy accumulation, sequential form" (meat-core/fact:meat): the published accumulation theorem, in the contracts' notation, for sequential channels with quantum memory under a product marginal constraint.
6. Theorem "Accumulated entropy bound under adaptive attacks" (meat-core/thm:accumulated-entropy): that theorem applied to the round channel: n times the single-round rate, less the penalty, on acceptance, in either source mode.
7. Theorem "Reduction of the rate to the convex program" (meat-core/thm:reduction): for an unconstrained declaration, the value $\underline{h}_\alpha$ bounds the single-round rate from below.
8. Theorem "Secrecy of the key from the accumulated entropy" (meat-core/thm:keylength): the key length displayed above.
9. Corollary "Key length from any lower bound on the value" (meat-core/cor:keylength-bound): any lower bound on $\underline{h}_\alpha$ gives a key length.
10. Theorem "The estimate and the first-order lower bound" (meat-core/frank-wolfe/thm:firstorder): the lower bound a run computes, and the estimate above it; the bound holds also at iterates that lie only in the *relaxed* feasible set, where the marginal condition and the declared rows hold within a window.
11. Corollary "Key length from the first-order bound" (meat-core/frank-wolfe/cor:fo-keylength): the key length from the first-order bound, clamped at zero.

With a non-empty constraint list the chain runs with the constrained rate and value in place of the unconstrained ones, under the attack-set

licence (Theorem "Constrained accumulation under the attack-set licence" (meat-core/thm:constrained-accumulation), Corollary "Key length from the constrained value" (meat-core/cor:constrained-key-length)); Theorem "The attack-set licence" (reductions/thm:licence) proves the licence for entries that carry spectral certificates. A run may also give up the entropy terms of chosen pairs of a sifted acceptance value and a joint sector, an Alice sector with a sector of the measured system, to compute a smaller value faster: the *dropped program* (Definition "Dropped set and dropped program" (meat-core/def:dropped)), whose first-order bound again bounds $\underline{h}_\alpha$ from below (Corollary "First-order bound for the dropped program" (meat-core/frank-wolfe/cor:fo-dropped)).

(b) The device models and what they certify

What the transmitter and the receiver establish about the declared devices, then how the reductions carry it to the finite model the key-rate chain starts from.

The source.

1. Theorem "Checkability and sufficiency of the source-replacement certificate" (transmitter/thm:certificate-sufficiency): the certificate's conditions are finitely many exact decisions, and under retention and the sequential round structure, measuring Alice's retained system before or after any attack gives the same state (source replacement).
2. Lemma "Certificate defined by an ensemble presentation" (transmitter/lem:ensemble-presentation): a certificate built from a virtual ensemble given as sector blocks, whose defining identities hold with no error term.
3. Lemma "Matched pinching of a block certificate" (transmitter/lem:matched-pinching): every state an admitted attack reaches from a block certificate is block diagonal and has Alice's declared marginal.
4. Proposition "Tagging channel for weights at or below the Poisson weights" (transmitter/decoy/prop:tagging): one channel, the same for every letter, maps each tagged virtual state to the phase-randomised weak coherent state it stands for, whenever the declared sector weights are at most the Poisson weights.
5. Corollary "The emitted decoy declaration" (transmitter/decoy/cor:emitted-decoy): the decoy source as the transmitter emits it, with an exact certificate and its identity with the physical source through the tagging channel.

The receiver.

6. Theorem "Click-pattern POVM" (receiver/thm:click-povm): the click patterns form a POVM, computed by inclusion and exclusion from the port Gram matrices, one positive operator per detector port whose quadratic form is the squared norm of the amplitude sent to that port; every element is block diagonal in the total photon number.
7. Theorem "Coarse-graining of the click-pattern POVM" (receiver/thm:coarse-graining): merging click patterns into the protocol's outcomes keeps a POVM; lower bounds on disjoint patterns add, and a merged outcome's bound says nothing of its parts.
8. Theorem "Multiclick bounds carried by the emitted weights" (receiver/thm:emitted-bound): for each probe setting (one, without a probe) a weight in $[0, 1]$ that bounds the multiclick element from below on every photon-number block above the cutoff.
9. Theorem "Detector efficiency in the port Grams" (receiver/detector-imperfections/thm:efficiency): an efficiency the same for every mode a port collects scales that port's Gram matrix, and it is applied exactly once.
10. Theorem "Dark counts on a photon-number block-diagonal POVM" (receiver/detector-imperfections/thm:dark-counts): dark counts raise a multiclick bound p_{mc} to $(1-r_{\min})\,p_{\mathrm{mc}}+r_{\min}$, where $r_{\min}$ is the least probability that they turn a pattern with at most one click into a multiclick one.
11. Theorem "Admissibility of the catalogue entries" (receiver/component-catalogue/thm:catalogue-admissibility): every catalogue entry is a contraction over its whole declared parameter range; only the generic block needs this checked when it is admitted.

From the devices to the finite model.

12. Theorem "Security transfer from a constrained squashed model" (reductions/thm:transfer): if the squashed inputs analysed contain the squashed image of every physical input, a bound ε on any security criterion of the squashed protocol holds for the physical one.
13. Theorem "The attack-set licence" (reductions/thm:licence): spectral certificates on the pull-backs of the carried operators place every squashed physical attack in the attack set of the constraint list; this is the attack-set licence.
14. Theorem "The two-sided reduction in trusted-source mode" (reductions/thm:two-sided): in trusted-source mode, source replacement and squashing compose, in either order, into one exact reduction.
15. Theorem "Exact emission of a reduction instance" (reductions/thm:emit): what a reduction instance emits is exactly admissible and

squashes the model it declares with no error term; where moving the data back onto the lattice changed them, the declared model is the moved one, as under *Emission* above.

16. Proposition "Flag-state squashers: existence, chain and encoded weights" (reductions/wpfss/prop:chain): with total certified weight below one, the receiver squashes both to the flag-state target and to the weight-preserving one, and a weight the latter encodes holds on every squashed state.

17. Corollary "Exact emission of the flat instance" (reductions/wpfss/cor:emitted-flat): the flag-state squasher as emitted for an optical receiver with no branch register, its weights rounded down and still certified.

(c) Statistics and the finite-size terms

1. Theorem "Validity of the admissible geometries" (statistics/thm:geometry-validation): an acceptance set that passes its validation predicate satisfies Definition "Admissible acceptance set" (meat-core/def:admissible-acceptance-set).

2. Theorem "Properties of the constructed box" (statistics/binomial-box/thm:box-construction): the box, one interval per outcome, contains every law of the honest envelope.

3. Theorem "The completeness bound" (statistics/binomial-box/thm:completeness-bound): an honest run fails the test with probability at most the completeness budget, with no independence across rounds assumed.

4. Theorem "Completeness of a singleton acceptance set" (statistics/singleton/thm:singleton-completeness): for a one-point acceptance set and independent honest rounds of one law, the probability of acceptance is multinomial.

5. Theorem "Acceptance support and the exact box oracle" (statistics/thm:box-oracle): exact linear minimisation over a box, with a dual witness of the same value.

6. Theorem "Accumulated entropy bound under adaptive attacks" (meat-core/thm:accumulated-entropy): the acceptance probability enters the entropy bound as the penalty $\alpha/(\alpha-1)\cdot\log_2(1/\Pr[\text{accept}])$.

7. Theorem "Secrecy of the key from the accumulated entropy" (meat-core/thm:keylength): the deductions for error correction, error verification and privacy amplification of the post-processing model (Definition "Fixed-length post-processing model" (meat-core/def:postprocessing-model)).

The honest model bears on completeness, rate and usability, never on the soundness of a key: the acceptance set is fixed before the run, and the observed frequencies only decide whether the run accepts.

(d) The assumptions a user checks

Each contract's front matter lists its main results with every assumption each rests on, and a table, *Assumptions at a glance*, of every assumption the contract states or its claims assume; the module names below open the contracts. We name here, with their short ids, the assumptions attested of the apparatus, the laboratories and the run, as opposed to those decided on declared numbers: every assumption a claim behind a key length declares. The Frank–Wolfe submodule also states Assumption "Exact linear minimisation over the feasible set" (meat-core/frank-wolfe/ass:lmo) (A-LMO), on which no claim rests.

1. Foundations: Assumption "The device measures the snapped family where the run reads the snap" (foundations/ass:snapped-measurement) (A-SNAP), where a run reads a measurement moved onto the lattice by a snap: on the blocks its record names, the device performs the moved measurement.

2. Transmitter: Assumption "Retention of the system A in trusted-source mode" (transmitter/ass:src) (A-SRC), in trusted-source mode; Assumption "Emission of the specified ensemble" (transmitter/ass:ens) (A-ENS), or for a decoy source Assumption "Emission of the declared weak-coherent-pulse states" (transmitter/decoy/ass:wcp) (A-WCP) in its place; Assumption "Sequential round structure" (transmitter/ass:seq) (A-SEQ).

3. Receiver: Assumption "Trusted and exactly characterised receiver" (receiver/ass:trust) (A-TRUST); Assumption "Completeness of the declared modes" (receiver/ass:modes) (A-MODES), the assumption a bright-illumination attack targets; Assumption "Passive linear optics" (receiver/ass:passive) (A-PASSIVE); Assumption "Threshold detectors without memory" (receiver/ass:thresh) (A-THRESH); Assumption "Mode-insensitive port response" (receiver/ass:port) (A-PORT); Assumption "Vacuum on undeclared input modes" (receiver/ass:vac) (A-VAC); Assumption "Device fixed independently of the adversary" (receiver/ass:nosel) (A-NOSEL); Assumption "Exact declared parameters" (receiver/ass:exact) (A-EXACT); Assumption "Round homogeneity of the instrument, probe and dark counts" (receiver/ass:round) (A-ROUND); and Assumption "Declared slack on the computed block eigenvalues" (receiver/ass:eig) (A-EIG), where the smallest eigenvalues of a finite window of photon-number blocks are computed rather than bounded analytically.

4. Reductions: Assumption "Supplier artefact whose claims hold of the device" (reductions/ass:recv) (A-RECV); Assumption "Each measuring party's device as one POVM per round" (reductions/ass:meas) (A-MEAS); Assumption "The station's private setting drawn from its own declared randomness" (reductions/ass:probe) (A-PROBE), where the receiver has a private setting; Assumption "Round homogeneity of the physical measurement" (reductions/ass:round) (A-ROUND-SQ); Assumption "Output-state test statistics" (reductions/ass:tsm) (A-TSM), that each round's public choices enter the test only through the recorded outcome; Assumption "Declared constants of the constraint list" (reductions/ass:constdecl) (A-CONST), with constraint entries, that their constants are fixed before the run by the device facts that certify them.

5. Statistics: Assumption "An acceptance value recorded in every round" (statistics/ass:alphabet) (A-ALPHABET), an acceptance value recorded in every one of the n rounds; Assumption "Honest rounds inside a declared envelope" (statistics/ass:hon-env) (A-HON-ENV), or Assumption "Honest i.i.d. rounds" (statistics/ass:hon) (A-HON) for a singleton set, which bear on completeness only; and Assumption "An honest law produced by a feasible state" (statistics/ass:reach) (A-REACH), that a feasible state produces statistics in the declared honest envelope (for a single declared law, that law), which no proof of secrecy uses but without which the declared devices do not describe the accepted statistics and a key length for them is not warranted.

6. Protocol compiler: Assumption "Specification faithful to the apparatus" (protocol-compiler/ass:spec) (A-SPEC); Assumption "Public choices as private coins of declared bias" (protocol-compiler/ass:coin) (A-COIN); Assumption "Round homogeneity of the declaration" (protocol-compiler/ass:rhom) (A-RHOM); Assumption "Alice's declared measurement where no source certificate is demanded" (protocol-compiler/ass:alice-meas) (A-ALICE-MEAS), where no source-replacement certificate is demanded and Alice has two or more letters; Assumption "Ideal qubit receiver on the Band-A route" (protocol-compiler/ass:ideal) (A-IDEAL), on the direct route only. Two more are obligations of whoever produced the reduction chain, Assumption "Soundness of every reduction link" (protocol-compiler/ass:link) (A-LINK) and, with a non-empty constraint list, Assumption "Attack-set licence" (protocol-compiler/ass:atk) (A-ATK). For a chain the reductions module emits, its theorems discharge A-LINK when the chain's status is exact, and A-ATK for the entries whose certificates it established (Remark "Discharge of the chain obligations by the reductions module" (protocol-compiler/rem:obligations-discharged)).

7. MEAT core:

- Assumption "Dedicated key register and one-way reconciliation" (meat-core/ass:keyreg) (A-KEYREG): the post-processing acts on the raw key the rounds write, and reconciliation runs one way from the key holder through a transcript of the fixed length λ_{EC};
- Assumption "Ideal two-universal privacy amplification" (meat-core/ass:hash) (A-HASH): privacy amplification uses an ideal two-universal family, and error verification a two-universal one;
- Assumption "Private, independent and fresh local randomness" (meat-core/ass:seed) (A-SEED): the post-processing seeds are drawn with their declared laws, privately, independently and afresh, and in trusted-source mode the source's letters and prepared systems are independent of the adversary, who never holds a purification of a prepared system;
- Assumption "Protocol parameters fixed before the run" (meat-core/ass:fixed) (A-FIXED): the acceptance set, n, ℓ, λ_{EC}, $\varepsilon_{\mathrm{PA}}$, $\varepsilon_{\mathrm{EV}}$, the authentication parameters and the Rényi order α are fixed before the run and applied unchanged, which is what makes the protocol fixed-length;
- Assumption "The implementation runs the analysed post-processing" (meat-core/ass:conform) (A-CONFORM): the implementation runs the analysed post-processing: it applies the acceptance test and the verification abort, sends no message beyond the model's, outputs the model's key or nothing, uses no key before the execution ends and acts on an authentication failure as prescribed;
- Assumption "Each party's laboratory is sealed" (meat-core/ass:lab) (A-LAB), at each party: its laboratory is sealed, so that nothing about its records leaves it but the key, the public messages and, in trusted-source mode at the source's party, the emitted systems, and the adversary acts on it only through the systems its declared measurements receive and the messages; it excludes every side channel, optical, electromagnetic, acoustic, power, timing or classical;
- Assumption "Authentication of the classical channel by a keyed scheme" (meat-core/ass:auth) (A-AUTH), for the composed security: every classical message goes through one authentication scheme that uses at most k_{auth} bits of pre-shared key and realises the authenticated channel within $\varepsilon_{\mathrm{auth}}$, a reused key's distance from an ideal one included;
- Assumption "Executions that share no key and no device state" (meat-core/ass:separate-executions) (A-SEPARATE), for adding the security parameters of several executions: they are fixed in advance, share no key, randomness or device state, leave their keys unused

until all have ended, and keep their premises with the others counted as the adversary's.

A first-order number adds no assumption about its solver: its linear minima are bounded in exact arithmetic (Lemma "A lower bound on the feasible-side minimum from any multipliers" (meat-core/frank-wolfe/lem:feasible-side-dual)).

The key-length theorem Theorem "Secrecy of the key from the accumulated entropy" (meat-core/thm:keylength) rests on A-KEYREG, A-HASH, A-SEED, A-FIXED, A-CONFORM, A-LAB at both parties and A-SEQ, and, through the accumulation theorem, on A-RHOM, in trusted-source mode on A-SRC and A-ENS, and with a non-empty constraint list on A-ATK. The composed security, Corollary "Security of the protocol and its net key" (meat-core/cor:composed-security), and with it a key length from any lower bound, Corollary "Key length from any lower bound on the value" (meat-core/cor:keylength-bound), adds A-AUTH; the sum over executions, Corollary "Security parameters of separate executions add" (meat-core/cor:separate-executions), adds A-SEPARATE. The receiver, reductions, compiler and statistics assumptions enter through the declaration a run is formed on, so which of them a key length rests on depends on its route, and the run record lists them. A key length on a reference companion rests in addition, through the reductions claims that build the companion, on A-MEAS, A-ROUND-SQ, A-CONST with constraint entries, and A-SNAP, the last only on the blocks its record names. No assumption covers the rounding of the first-order bound as a run evaluates it (Remark "Exact real arithmetic in the first-order bound" (meat-core/frank-wolfe/rem:fo-arithmetic)): the key-length statement is about the exact bound, and only a certified number removes that gap.

Main results, module by module

Each contract's main results, the modules in dependency order (a module after every module it builds on). A heading opens the statement in its contract.

Foundations: quantum-information preliminaries, number model, exact admissibility and emission

version 0.12.0, statements digest

Lemma 1.4 (Extended divergence kernel)
The divergence kernel $d(u, v) = u \log_2(u/v)$, extended to vanishing arguments, is jointly convex, lower semicontinuous and non-increasing in v. These properties keep the objective well defined at support boundaries and let it be composed with concave branch quantities.

Lemma 1.5 (Pinching lowers the trace power)
For $\alpha > 1$, pinching the α-th root of a positive semidefinite operator and raising the result to the power α never increases the trace, and leaves it unchanged exactly when the operator is block diagonal for the pinching. This bounds the trace-power terms of the MEAT objective.

Lemma 3.4 (Exactness of a sum that rounds nowhere)
A sequential sum rounds nowhere exactly when every exact partial sum is a binary64 number, and then the computed sum equals the exact sum at every step. A predicate stated on such a sum is a predicate on the declared values themselves.

Definition 4.3 (Exact admissibility of a declared operator family)
A declared operator family is exactly admissible for a sector map with constant c when (T) its own sector map agrees, (H) every block is exactly Hermitian, (P) positive, (C) it sums to c times the identity and (S) its declared-order sum rounds nowhere. Each is a property of the declared numbers.

Requirement 9.1 (Exact admissibility at every module boundary)
Declared operator data cross every module boundary exactly admissible, so every sum a predicate reads is the exact sum. The declarer makes them so, or, for data an instance passes on unchanged, the consumer's exact decision on entry does, and a consumer that finds otherwise refuses them.

Receiver POVM (receiver)

version 0.11.0, statements digest

Theorem 4.5 (Click-pattern POVM)
The click patterns of a receiver instrument form a POVM whose elements follow from the port Grams by inclusion and exclusion. Every element is block diagonal in the total input photon number, and vacuum gives only the no-click outcome.

Theorem 5.6 (Coarse-graining of the click-pattern POVM)
The emitted POVM of a declared coarse-graining is a POVM, block diagonal in the photon number, and lower bounds on disjoint raw outcomes add. A bound on a merged outcome gives nothing about the click patterns it merges.

Theorem 8.14 (Multiclick bounds carried by the emitted weights)
Each setting's emitted weight bounds its multiclick element on every photon-number block above the cutoff, and their minimum does so at every setting, under the eigenvalue-slack assumption A-EIG only where computed block eigenvalues enter. Each weight lies in [0, 1] and is finitely computable.

Proposition 8.17 (Range of the strengthening-slot weights)
At every setting the strengthening slot's weights (high-photon lower bounds on disjoint sets of raw outcomes) are non-negative and sum to at most one,

as are their minima over the settings, and each emitted multiclick weight lies in [0, 1]. That the emitted weights are valid bounds is Theorem 8.14.

Theorem 16.3 (Admissibility of the catalogue entries)
Every catalogue entry maps its whole declared parameter domain to contractions on its modes, lossless exactly on the subfamily the catalogue's table lists for it. Only the generic block, an arbitrary matrix on declared modes, needs its contractivity checked at admission.

Theorem 18.1 (Detector efficiency in the port Grams)
Efficiency η_k uniform on detector port k scales that port's Gram by η_k, keeping its rank if η_k > 0; varying within a port, it is no scalar and can lower the rank. It enters only as the compiled instrument matrix's last layer, and the loss decomposition Lemma 4.2 checks that loss is counted once.

Theorem 19.2 (Dark counts on a photon-number block-diagonal POVM)
A dark-count channel keeps the photon-number block-diagonal POVM structure and never moves weight out of the multiclick set. It raises the multiclick bound p_mc to $(1 - \text{r_min})\text{p_mc} + \text{r_min}$, where r_min is the least multiclick mass over every non-multiclick column, the no-click column included.

Transmitter source model (transmitter)

version 0.13.0, statements digest

Lemma 3.4 (Certificate defined by an ensemble presentation)
A virtual ensemble in sector blocks defines, by canonical purification and a square-root measurement, a block certificate with pure blocks that exactly reproduces Alice's marginal and, under that measurement, the ensemble. Only her marginal, a transposed sum of declared blocks, is formed as numbers.

Theorem 4.1 (Checkability and sufficiency of the source-replacement certificate)
In any form (pure projector, sector blocks, ensemble presentation) the certificate's algebraic conditions are finitely many exact decisions making Alice's measurement reproduce the virtual ensemble. Under retention and sequential rounds she may measure before or after any attack: source replacement.

Lemma 4.5 (Matched pinching of a block certificate)
A block certificate is unchanged by pinching Alice's sectors or the virtual signal's sectors paired with them, so every state an admitted attack reaches from it is block diagonal with Alice's declared marginal. An operator merely constrained to have that marginal need not be block diagonal.

Proposition 8.1 (Tagging channel for weights at or below the Poisson weights)
One channel, independent of the label, maps each tagged virtual state to its phase-randomised weak coherent state whenever the declared sector weights lie at or below the Poisson weights. The weights therefore need only a certified inequality, never an exact Poisson weight.

Corollary 9.2 (The emitted decoy declaration)
The emitted decoy declaration has an exact certificate against the tagged ensemble, its source identity by the tagging channel, and block-diagonal attacked states. Exactness is decided on products of declared weights and formed dyad entries; lowering a weight gives another tagged declaration.

Proof reductions by squashing (reductions)

version 0.22.0, statements digest

Theorem 3.11 (Security transfer from a constrained squashed model)
If the analysed squashed inputs contain the squashed image of every physical input, a bound ε on any criterion of the output state of the squashed protocol holds for the physical protocol with the same ε. No property of the criterion is used.

Theorem 6.7 (The attack-set licence)
If each operator the squashing carries pulls back to a physical operator $\leq$, $=$ or $\geq$ a multiple of the identity, every squashed physical attack respects the resulting constraint list in either source mode, factor by factor for per-factor bounds. Security against such attacks transfers with the same ε.

Theorem 8.6 (The two-sided reduction in trusted-source mode)
In trusted-source mode the source map and the squashing reduction compose, in either order, into one exact operational reduction into the admissible attack set. Security of the virtual squashed model thus transfers with the same ε to the specified physical protocol, against attacks of its type.

Theorem 11.2 (Exact emission of a reduction instance)
The emitted target is computed exactly, decided exactly admissible before emission, and squashes the declared model exactly; nearness to the handed model bounds nothing. On blocks the record names as resting on A-SNAP, the run's conclusions about the device rest on that premise.

Corollary 12.1 (The reference companion of a reduction instance)
The same instance, run on the reference in place of the snap, or on its own snapped blocks where none is handed, is a reduction instance of the emitted shape that squashes that model exactly. Its entries are enclosed from the reference, requested ones where their certificates hold there.

Proposition 19.4 (Flag-state squashers: existence, chain and encoded weights)
With total certified weight below one, the physical POVM squashes to both targets, and the squashers compose exactly through a flag-noise channel. A weight the weight-preserving target encodes holds on every squashed state; under the flag-state target it holds only on reachable ones.

Corollary 21.3 (Exact emission of the flat instance)
The flat instance's emitted family is exactly admissible and pulls back exactly to the declared model on the channel system, its weights rounded down yet certified. Where the normalizer moved a block below the cutoff, nearness to the unrounded family bounds nothing.

Theorem 23.2 (The noise channel of a domination)
If G dominates F sector by sector, G squashes exactly to F with added flag sectors, or to F itself where F has flag sectors, through a noise channel that keeps each sector with probability one minus its weight. The adjoint returns each sector projector scaled by that probability.

Protocol compiler (protocol-compiler)

version 0.24.0, statements digest

Theorem 4.4 (Encoding of public choices in the measurement operators)
A public choice of known bias enters as weights inside the measurement operators: the family stays a POVM, fixed weights are reproduced exactly as choice probabilities where the weighted family is emitted, and setting-dependent weights commute with the chain. A fraction outside them is refused.

Theorem 6.2 (Soundness, completeness and decidability of the validator)
Passing the validator's eight checks establishes every target condition of a well-formed declaration, and every check but the feasibility check is complete, so a well-formed declaration is refused only when no default or supplied candidate is verified feasible. Every decision is finite and exact.

Theorem 9.4 (The emission gates and their outcomes)
Each gate is decidable or a presence test, and each failure has one fixed outcome, a refusal or an emission flagged as backing no key length, never a default, dropped entry, changed source mode or licence the compiler writes. Passing asserts nothing about the apparatus or attack-set inclusion.

MEAT key-rate engine (meat-core)

version 1.32.0, statements digest

Theorem 2.10 (Accumulated entropy bound under adaptive attacks)
On acceptance, the raw key's Rényi entropy given the adversary's information is at least n times the single-round rate less $\alpha/(\alpha-1)\cdot\log_2(1/\Pr[\text{accept}])$, for any adaptive attack with a finite-dimensional memory. Trusted-source mode reads the product marginal of Lemma 2.9; untrusted mode needs none.

Theorem 5.4 (Reduction of the rate to the convex program)
For a declaration that adds no constraints on the state (an empty constraint list), in either source mode, the value h_low of the finite convex program bounds the single-round rate from below. Every step of the reduction is an equality or lowers the bound, so the computable program is conservative.

Theorem 10.7 (Secrecy of the key from the accumulated entropy)
For a Rényi order α in (1, 2), a key of length $\ell \le \text{n}\cdot\text{h_low} - \lambda\text{_EC} - \lceil\log_2(1/\varepsilon\text{_EV})\rceil - \alpha/(\alpha-1)\cdot\log_2(1/\varepsilon\text{_PA}) + 1$ is ε_PA-secret on the accepted state at every all-at-once finite-register attack the reduction chain carries onto the round model (A-LINK). Corollary 10.8 extends it to every attack.

Corollary 10.8 (Security of the protocol and its net key)
Secrecy and correctness compose to (ε_PA + ε_EV)-security with honest authentication; over an authentication scheme within ε_auth that consumes k_auth bits of pre-shared key, the protocol is (ε_PA + ε_EV + 2ε_auth)-secure; when both parties output a key, its net key gain is ℓ − k_auth bits.

Corollary 10.18 (Key length from any lower bound on the value)
Any h ≤ h_low may replace h_low in the key-length bound through max{0, h}, with the same security statement. This is where every solver obtains its key length; a bound h ≤ 0 yields no positive key when ε_EV < 1.

Theorem 19.1 (The estimate and the first-order lower bound)
In exact real arithmetic the best linearisation over the iterates, even ones feasible only up to windows, bounds h_low from below given positive probabilities, maximal branch support, unperturbed gradients and bounds on both linear minima. An exactly feasible point's objective bounds it from above.

Corollary 19.6 (First-order bound for the dropped program)
In either source mode the first-order bound holds for the dropped program (Definition 9.2), with maximal support asked only of key-active pairs (acceptance value, joint sector) whose entropy is kept. It bounds h_low from below, also at iterates feasible only up to windows.

Corollary 19.7 (The first-order bound on the reference companion)
At any points, such as those of a search on the snapped declaration or of one continued on the companion, the bracket formed with the reference companion's objective, gradient, entries and linear minima bounds the companion's value from below; its key length is the reference model's.

Corollary 21.3 (Key length from the certified bracket)
The largest bracket formed by Lemma 21.1 over finitely many points is a first-order bound of Theorem 19.1, so it is at most the value of the declaration it is formed on and gives that declaration's fixed-length key length exactly as h_lb does.

Acceptance statistics (statistics)

version 0.10.0, statements digest

Theorem 1.5 (Validity of the admissible geometries)
An acceptance set whose data pass validation (a probability vector, or a feasible linear system) is a non-empty, compact, admissible acceptance set (Definition 2.5). For a box of per-outcome bounds, validation means the lower sum to at most one and the upper to at least one.

Theorem 3.1 (Acceptance support and the exact box oracle)
Over a non-empty canonical box, filling the coordinates of smallest coefficient first minimises a rational linear functional exactly, with a dual witness of the same value. The outcomes with a positive upper bound include every outcome on which a point of the box puts weight, and can include more.

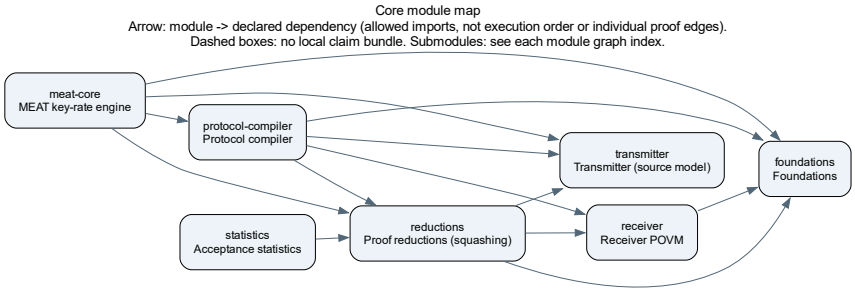
Theorem 9.2 (Properties of the constructed box)
For any admissible endpoint rule (count bounds meeting binomial tail budgets), the acceptance box has exact rational bounds and contains every law within the honest envelope's bounds. It is non-empty and compact and, by construction, an admissible acceptance set (Definition 2.5).

Theorem 10.2 (The completeness bound)
If each honest round's conditional outcome law stays inside the envelope, an honest run fails the acceptance test with probability at most ε^*, the sum of the binomial tails at the envelope's ends beyond the box's count bounds, and ε^* is at most the completeness budget. No independence is needed.

Theorem 18.1 (Completeness of a singleton acceptance set)
Under i.i.d. honest rounds a singleton acceptance set accepts with its point's multinomial probability. That is 0, with no completeness budget below one, if n times a coordinate is no integer or the point leaves the law's support; else the least budget is one minus it, zero only for one point mass.

The module map

An arrow points from a module to a module it may build on (its declared dependencies, not individual proof edges); a dashed box is a module without claims yet.


Core module map
Arrow: module -> declared dependency (allowed imports, not execution order or individual proof edges).
Dashed boxes: no local claim bundle. Submodules: see each module graph index.
meat-core
MEAT key-rate engine
protocol-compiler
Protocol compiler
transmitter
Transmitter (source model)
foundations
Foundations
statistics
Acceptance statistics
reductions
Proof reductions (squashing)
receiver
Receiver POVM